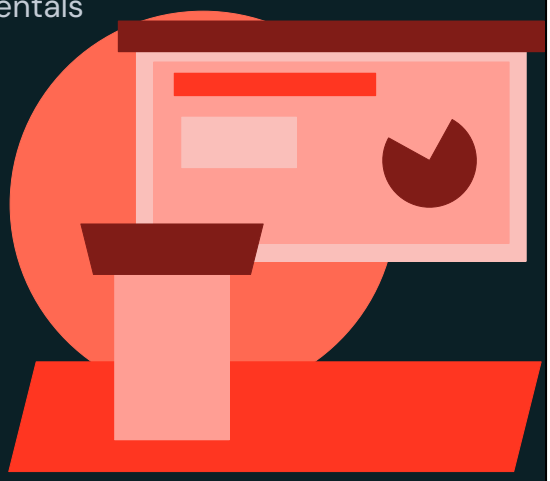
databricks

Lakeflow Spark Declarative Pipeline Fundamentals

LECTURE

Ensure Data Quality with Expectations



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn how to create and add data quality expectations to your pipelines, apply expectations to validate streaming data, and understand available actions for handling records that fail data quality checks.

Ensure Data Quality with Expectations

Creating Expectations

With Lakeflow Spark Declarative Pipelines you can apply **data quality rules** to validate rows during ETL processes to gain insights into data integrity using the following

```
CONSTRAINT constraint-name EXPECT (column condition) [ON VIOLATION action]
```

Action	SQL Syntax Example	Result
WARN (default)	`CONSTRAINT valid_notification EXPECT (notifications IN ('Y','N'))`	• Invalid rows **are still written** to the target • Logs include **counts of valid vs. invalid records** and other metrics
DROP	`CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01")` `ON VIOLATION DROP ROW`	• Invalid rows **dropped** from table • The count is logged alongside other metrics
FAIL	`CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL)` `ON VIOLATION FAIL UPDATE`	• Causes a **failure of a single flow** and does not cause other flows in your pipeline to fail. • **Manual** intervention is required

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Lakeflow Spark Declarative Pipelines lets you apply data quality rules called expectations during your ETL processes. These rules validate your data row-by-row to ensure integrity.

The general syntax is: CONSTRAINT <constraint-name> EXPECT (<column condition>) [ON VIOLATION <action>]

You can specify three types of actions when a violation occurs:

1. WARN (default)

- Logs a warning but still writes invalid rows to the target.
- Example: To warn if the **notifications** column contains values other than 'Y' or 'N': CONSTRAINT valid_notification EXPECT (notifications IN ('Y','N'))
- Logs will include counts of valid vs. invalid records and other metrics, helping you monitor data quality without stopping the pipeline.

2. DROP

- Drops invalid rows from the output.
- Example: To drop rows where **order_timestamp** is before "20201-01-01": CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01") ON VIOLATION DROP ROW
- Invalid rows are excluded, and counts of dropped rows are logged for auditing.

3. FAIL

- Fails the specific flow if the rule is violated, requiring manual intervention. Other flows in the pipeline continue running.
- Example: To fail if **customer_id** is NULL: CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION FAIL UPDATE
- This helps catch critical issues that must be resolved before proceeding.

Ensure Data Quality with Expectations

Adding Data Quality Expectations Example

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
 (
   CONSTRAINT valid_notifications EXPECT (notifications IN ('Y','N')),
   CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01") ON VIOLATION FAIL UPDATE,
   CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW
 )
AS
SELECT
   Order_id,
   timestamp(order_timestamp) AS order_timestamp,
   Customer_id,
   notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

WARN in metrics

DROP the row

FAIL THE PIPELINE

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

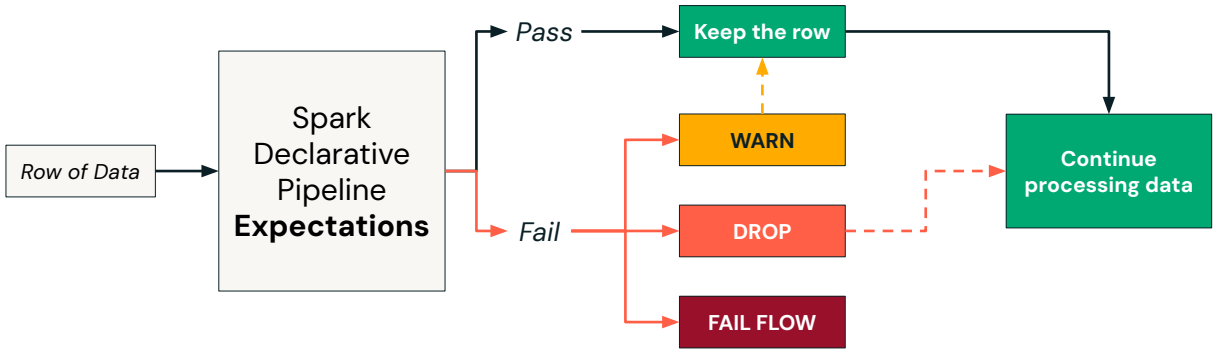
Let's look at an example of how to add constraints in Declarative Pipelines.

Here we will create a streaming table named orders_silver. This streaming table will have the following three constraints:

- WARN - CONSTRAINT valid_notifications EXPECT (notifications IN ('Y','N')),
- FAIL - CONSTRAINT valid_date EXPECT (order_timestamp > "2021-01-01") ON VIOLATION FAIL UPDATE,
- DROP - CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,

Ensure Data Quality with Expectations

Actions Overview



As of 2025Q2 materialized views that use **expectations** are **always fully refreshed**

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now let's look at how expectations are applied within the pipeline architecture.

Here's what happens step by step:

1. A row of data enters the pipeline.

1. That row is evaluated against any defined expectations (constraints).

- If the row passes all expectations, it's kept, and the pipeline continues processing as normal.

- If the row fails an expectation, the action defined in the constraint determines what happens next:
  - WARN (default): The failure is logged, the row is still included, and the pipeline continues.
  - DROP: The row is discarded, but the pipeline continues processing remaining data.
  - FAIL: The pipeline fails immediately for that specific flow, and manual intervention is required. Other flows are unaffected.

As of Q2 2025, any materialized views that use expectations will always be fully refreshed during pipeline runs.

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Always close with a thank you for participating in the course or watching this training video.